```
template<typename T>
std::list<T> sequential_quick_sort(std::list<T> input)
{
    if(input.empty())
    {
        return input;
    }
    std::list<T> result;
    result.splice(result.begin(),input,input.begin());          ❶
    T const& pivot=*result.begin();                             ❷

    auto divide_point=std::partition(input.begin(),input.end(),
            [&](T const& t){return t<pivot;});                  ❸

    std::list<T> lower_part;
    lower_part.splice(lower_part.end(),input,input.begin(),
        divide_point);                                          ❹

    auto new_lower(
        sequential_quick_sort(std::move(lower_part)));          ❺
    auto new_higher(
        sequential_quick_sort(std::move(input)));               ❻

    result.splice(result.end(),new_higher);                     ❼
    result.splice(result.begin(),new_lower);                    ❽
    return result;
}
```

Although the interface is FP-style, if you used FP-style throughout you'd do a lot of copying, so you use "normal" imperative style for the internals. You take the first element as the pivot by slicing it off the front of the list using `splice()` ❶. Although this can potentially result in a suboptimal sort (in terms of numbers of comparisons and exchanges), doing anything else with a `std::list` can add quite a bit of time because of the list traversal. You know you're going to want it in the result, so you can splice it directly into the list you'll be using for that. Now, you're also going to want to use it for comparisons, so let's take a reference to it to avoid copying ❷. You can then use `std::partition` to divide the sequence into those values *less than* the pivot and those *not* less than the pivot ❸. The easiest way to specify the partition criteria is to use a lambda function; you use a reference capture to avoid copying the `pivot` value (see appendix A, section A.5 for more on lambda functions).

`std::partition()` rearranges the list in place and returns an iterator marking the first element that's *not* less than the pivot value. The full type for an iterator can be quite long-winded, so you just use the `auto` type specifier to force the compiler to work it out for you (see appendix A, section A.7).

Now, you've opted for an FP-style interface, so if you're going to use recursion to sort the two "halves," you'll need to create two lists. You can do this by using `splice()` again to move the values from `input` up to the `divide_point` into a new list: `lower_part` ❹. This leaves the remaining values alone in `input`. You can then sort the two lists with recursive calls ❺, ❻. By using `std::move()` to pass the lists in, you can avoid copying here too—the result is implicitly moved out anyway. Finally, you can use `splice()` yet again to piece the `result` together in the right order. The `new_higher` values go on the end ❼, after the pivot, and the `new_lower` values go at the beginning, before the pivot ❽.